A 16-Bit-Native Entropy Coding Pipeline for Lossless Medical Image Compression

Kuldeep Singh

Abstract—Lossless compression is essential in medical imaging, where archival storage and diagnostic workflows require exact pixel reconstruction from 10–16-bit-per-sample data. Most entropy coders target 8-bit byte streams; medical images break this assumption by producing residual alphabets of thousands of symbols after spatial prediction. This paper presents MIC, a lossless codec for Digital Imaging and Communications in Medicine (DICOM) images that operates natively on 16-bit residuals. MIC combines spatial prediction, 16-bit run-length encoding, and large-alphabet Finite State Entropy (FSE), a table-driven implementation of asymmetric numeral systems (ANS). Rather than splitting 16-bit pixels into byte pairs before entropy coding, MIC treats each predicted residual as a single 16-bit symbol, capturing correlations between the high and low byte directly—a structural advantage quantified at 0.3–1.1 bits per symbol on the test dataset. The implementation extends FSE to support up to 65,535-symbol alphabets and includes a multi-state interleaved decoder that exploits instruction-level parallelism (ILP) during decompression. Evaluation on 21 de-identified DICOM images spanning nine modalities shows the 16-bit-native pipeline improves compression ratio by 5–22% (geometric mean 14%) over a Delta+Zstandard baseline, reaching a geometric-mean ratio of 3.12 times—within 1% of High-Throughput JPEG 2000 (HTJ2K, 3.15 times). A four-state decoder yields a 45% geometric-mean FSE stage decompression speedup on ARM64. Single-threaded, MIC's C decoder is faster than HTJ2K on every ARM64 image and on 16 of 21 AMD64 images; a strip-parallel mode adds near-linear scaling for large images. A 20-kilobyte pure JavaScript decoder and a Go WebAssembly build enable client-side in-browser decoding, confirming that 16-bit-native entropy coding is a practical design point where throughput, simplicity, and deployment portability matter.

Index Terms—medical image compression, lossless compression, entropy coding, asymmetric numeral systems (ANS), finite state entropy (FSE), DICOM, high-bit-depth imaging.

I. Introduction

MEDICAL imaging systems routinely generate large volumes of high-bit-depth data. Modalities such as computed tomography (CT), magnetic resonance imaging (MR), digital radiography (computed radiography, CR; plain X-ray, XR), mammography, and tomosynthesis commonly store pixel values at 10, 12, or 16 bits per sample. A single digital breast tomosynthesis (DBT) acquisition view produces 60 or more reconstructed slices, totalling over 600 MB of raw pixel data per view. A complete bilateral four-view screening study exceeds 2 GB uncompressed. Efficient lossless compression is essential for archival storage, network transmission, and real-time rendering in clinical Picture Archiving and Communication Systems (PACS) and diagnostic viewers.

DICOM [1] supports several established lossless transfer syntaxes, including JPEG 2000, HTJ2K, JPEG-LS (a lossless/near-lossless predictive standard), and Run-Length Encoding (RLE) Lossless. These methods offer different tradeoffs among compression ratio, decoding speed, and implementation complexity [24], [25]. JPEG 2000 and HTJ2K provide strong compression performance but rely on transform and block-coding pipelines that are comparatively complex. JPEG-LS uses predictive coding and often yields strong lossless ratios, but its throughput characteristics remain application-dependent. RLE Lossless is simple but typically provides limited compression because it operates on byte-oriented runs and does not include an effective decorrelation stage.

This work studies a different design point for lossless medical image compression: direct coding of predicted 16-bit residual streams. Throughout this paper, a residual stream denotes the sequence of per-pixel prediction errors $r_{i,j} = x_{i,j} - \hat{x}_{i,j}$ produced by a spatial predictor, kept at the native 16-bit precision of the source pixels rather than re-serialised into bytes. As a concrete example: if the predicted pixel value is 1020 and the true pixel is 1023, the residual is +3. Smooth regions therefore produce many residuals near zero, and these near-zero residuals recur frequently—properties that motivate a 16-bit-native run-length and entropy coding stage.

The central observation is that the dominant entropy coders in modern compression systems, including Huffman coding, arithmetic coding, Lempel-Ziv 1977 (LZ77), and FSE/ANS as used in Zstandard, were designed primarily for 8-bit byte streams. Medical images break this assumption: DICOM stores pixels at 10–16 bits per sample, yielding symbol alphabets of 1,024 to 65,536 entries. Even after spatial delta prediction, the residual alphabet for a 16-bit CT image can span tens of thousands of distinct values. An entropy coder that assumes an 8-bit alphabet must either re-encode 16-bit symbols as byte pairs (losing inter-byte correlation and doubling the number of coding steps) or truncate the alphabet at 256 and fall back to raw storage for rare symbols. This mismatch is structurally costly: for example, small signed residuals such as $-2, -1, 0, +1$ have highly predictable high bytes, so encoding the high and low bytes independently forces the coder to spend bits on information already implied by the other byte. On our test images, the mutual information $I(X_H; X_L)$

K. Singh is with Innovaccer Inc. E-mail: kuldeep.singh@innovaccer.com. ORCID: 0009-0004-8476-0118.
Code: https://github.com/pappuks/medical-image-codec.

between the high and low bytes of delta residuals ranges from 0.3 bits/symbol (MR) to 1.1 bits/symbol (MG3), corresponding to 8–22% of total entropy and explaining MIC's 5–22% empirical advantage over Delta + Zstandard.

Based on this observation, we present MIC, a lossless codec that combines:

1) a simple spatial predictor,
2) a 16-bit-native run-length representation of the residual stream, and
3) an extended FSE/tANS (table-based ANS) entropy coder supporting large active symbol alphabets.

The codec also includes multi-state interleaved entropy decoding to improve decompression throughput by increasing ILP.

A. Contributions

The main contributions of this paper are as follows.

1) A 16-bit-native lossless coding pipeline for medical images. We present a Delta + 16-bit RLE + extended FSE pipeline for grayscale medical image compression that consistently outperforms Delta + Zstandard on all 21 tested images, with the improvement attributable to the structural cost of byte-splitting 16-bit residuals.
2) An extended table-based entropy coder for large residual alphabets. The proposed implementation supports up to 65,535 distinct symbols. One 16-bit code value ($2^{16} - 1 = 65535$) is reserved as the overflow delimiter for full-range images, leaving up to 65,535 codes available for entropy coding. Tables are dynamically sized to the observed symbol range, reducing working-set size from 512 KB to approximately 2 KB for 8-bit images.
3) A multi-state interleaved entropy decoder. We implement and evaluate two-state and four-state FSE decoding to exploit ILP, achieving a geometric-mean +45% FSE stage speedup on ARM64 (+43% on AMD64) with no compression ratio penalty.
4) An empirical evaluation against standard codecs and a general-purpose baseline. We compare MIC with HTJ2K (OpenJPH) [29], JPEG-LS (CharLS) [30], and Delta + Zstandard on a dataset of 21 DICOM images spanning nine modalities (described in Section V).
5) A compact JavaScript/WebAssembly (WASM) decoder implementation. We provide a ∼20 KB pure JavaScript (JS) decoder and a Go WebAssembly build to explore deployment feasibility for client-side viewing.

B. Scope

This paper is scoped to lossless single-frame grayscale medical image compression. Multi-frame and RGB extensions are outside the scope of this work. The present work does not address lossy coding, progressive decoding, or region-of-interest coding.

C. Organization

Section II reviews related work and positions MIC relative to existing methods. Section III describes the proposed coding pipeline. Section IV presents the multi-state entropy decoder. Section V describes experimental methodology. Section VI reports compression and speed results. Section VII concludes.

II. Related Work

A. Lossless Compression in Medical Imaging

JPEG 2000 [2] employs the reversible 5/3 wavelet transform [20], [23] with Embedded Block Coding with Optimized Truncation (EBCOT), achieving excellent ratios at significant decompression overhead. HTJ2K [3], [27] replaces EBCOT with a faster block coder, substantially improving decode speed. JPEG-LS [4], [5] uses context-adaptive Median Edge Detector (MED) prediction with Golomb-Rice coding; it is an important lossless baseline as a standardized predictive codec widely deployed in medical PACS. RLE Lossless (DICOM TS 1.2.840.10008.1.2.5) applies byte-level PackBits without decorrelation, yielding poor ratios (typically <1.5×). Higher-complexity approaches such as the Context-based Adaptive Lossless Image Codec (CALIC) [18] and the Free Lossless Image Format (FLIF) [19] achieve strong ratios on natural images but lack DICOM standardization and browser decoders.

These codecs are the primary baselines for this work. The goal is not to supersede them but to investigate a simpler residual-domain design specifically targeting high-bit-depth images with strong throughput and deployment portability.

B. Entropy Coding and the Byte Assumption

Asymmetric numeral systems (ANS) [7], [8] replace arithmetic coding's interval subdivision with integer state transitions. Finite State Entropy (FSE) [9] is a table-driven tANS implementation with $\mathcal{O}(1)$ per-symbol operations, popularized by Zstandard [10]. Recent work has formalized ANS efficiency bounds [12] and statistical interpretations [13]. In practice, Huffman coding becomes impractical above ∼4,096 symbols and FSE in Zstandard is capped at 4,096; modern formats such as JPEG XL [6] likewise target byte streams.

Information-theoretic cost of byte-splitting. For a 16-bit symbol X decomposed into high byte X_H and low byte X_L, a byte-level coder achieves at best $H(X_H)+H(X_L) = H(X) + I(X_H; X_L)$. After delta prediction, when a residual is small (for example, ±30), X_H is nearly deterministic given X_L, but it must still be encoded independently. On our test images, $I(X_H; X_L)$ ranges from 0.3 bits/symbol (MR) to 1.1 bits/symbol (MG3), corresponding to 8–22% of total entropy and closely matching MIC's empirical advantage over Delta + Zstandard.

C. ANS and Interleaved Decoding

Prior work has shown that interleaving or multi-stream organization can improve ANS decoder throughput. Giesen [14], [15] demonstrated the value of multi-stream range ANS (rANS); Lin et al. [16] extended parallel rANS to decoder-adaptive scalability; Weissenberger and Schmidt [17] showed massively parallel ANS decoding on GPUs. The present work does not claim novelty in ANS itself or in interleaving as a general concept. Rather, the contribution lies in applying these ideas in a 16-bit-native medical image codec, together with a specific large-alphabet implementation and throughput-oriented software design.

D. Positioning of This Work

Table I distinguishes MIC from prior work across five dimensions relevant to medical image compression.

TABLE I: Feature comparison of MIC with related codecs and entropy coding methods.

Feature	JPEG-LS	HTJ2K	Zstd	rANS	MIC
Lossless medical	Yes	Yes	General	General	Yes
Native 16-bit	No	No	No	Configurable	Yes
Multi-state ILP	No	No	No	Yes	Yes
Browser decoder	No	No	Partial	No	Yes
Source lines	5k	20k	N/A	N/A	1.1k

III. Proposed Method

Each stage of the pipeline described in Section I is simple enough to decode in a single sequential pass with minimal branching, as illustrated in Fig. 1.

A. Spatial Prediction

Let $x_{i,j}$ denote the pixel at row i, column j. For interior pixels, MIC predicts the current pixel from the average of the top and left neighbors:

$$\hat{x}_{i,j} = \left\lfloor \frac{x_{i-1,j} + x_{i,j-1}}{2} \right\rfloor \tag{1}$$

where the division is integer division rounded down (matching the decoder exactly), and all arithmetic is performed in unsigned integers. The residual is formed as:

$$r_{i,j} = x_{i,j} - \hat{x}_{i,j}. \tag{2}$$

Boundary pixels use only the available neighbor, and the first pixel is stored directly. This predictor was selected because it is computationally simple, branch-light in decoding, and effective on the current dataset.

Predictor comparison. We implemented and compared four predictors through the full RLE + FSE pipeline on the 21-image dataset: left-only (left neighbor, no top), average (MIC default), Paeth (PNG predictor [26]), and MED (JPEG-LS [4]). Table II summarizes geometric means and relative gains.

MED decompression is 1.5–2× slower due to the three-way conditional branch and diagonal neighbor dependency that prevents the branch-free interior loop used by the average predictor. MED yields the best geomean ratio (+1.6% over Avg) but at significant speed cost; the average predictor is retained as the default for its consistent per-modality performance and branch-free decompression path.

TABLE II: Predictor ablation summary: geometric means and win counts across all 21 images.

Predictor	Geo. Mean	Wins	Avg→X
Left-only	3.38×	3/21	−2.3%
Average (MIC)	3.46×	13/21	baseline
Paeth	3.48×	2/21	+0.5%
MED (JPEG-LS)	3.52×	16/21	+1.6%

B. Effective Bit Depth and Overflow Coding

For 16-bit images, residuals can span $[-65535, +65535]$. Rather than widening the main residual stream to 32 bits, MIC uses an overflow delimiter scheme derived from the effective image bit depth:

$$d = \lceil \log_2(\max(x) + 1) \rceil. \tag{3}$$

Residuals within an in-range interval are mapped directly to 16-bit codes. Residuals outside that interval are encoded using a delimiter followed by the raw pixel value.

Encoding procedure:

1) Compute deltaThreshold $= 2^{d-1} - 1$.
2) Compute delimiter $D = 2^d - 1$.
3) For each residual $r_{i,j}$: if $|r_{i,j}| <$ deltaThreshold, emit uint16(deltaThreshold + $r_{i,j}$); otherwise emit delimiter D, then emit raw pixel $x_{i,j}$.

The threshold condition is strict ($<$, not $\leq$), so residuals with $|r| =$ deltaThreshold take the overflow path. Residual zero maps to code value deltaThreshold exactly. Table III illustrates the mapping for a representative bit depth.

TABLE III: Overflow coding example for $d = 12$ bits (deltaThreshold= 2047, $D = 4095$).

Residual r	Emitted code(s)	Notes
−2	2045	in-range
−1	2046	in-range
0	2047	in-range (= threshold)
+1	2048	in-range
+2	2049	in-range
±2047	4095, raw pixel	overflow

Non-collision proof. In-range residuals satisfy $|r| \leq$ deltaThreshold $- 1$, so stored values range from 1 to $2^d - 3$. The delimiter is $D = 2^d - 1$. Since $2^d - 3 < 2^d - 1$, the stored value of a direct residual can never equal D; the two ranges are disjoint.

Decoding procedure: Read the next uint16 value c. If $c \neq D$, recover the residual as $r =$ int16($c-$ deltaThreshold) and reconstruct $x_{i,j} = \hat{x}_{i,j} + r$. If $c = D$, the raw pixel follows: read the next uint16 as $x_{i,j}$ directly.

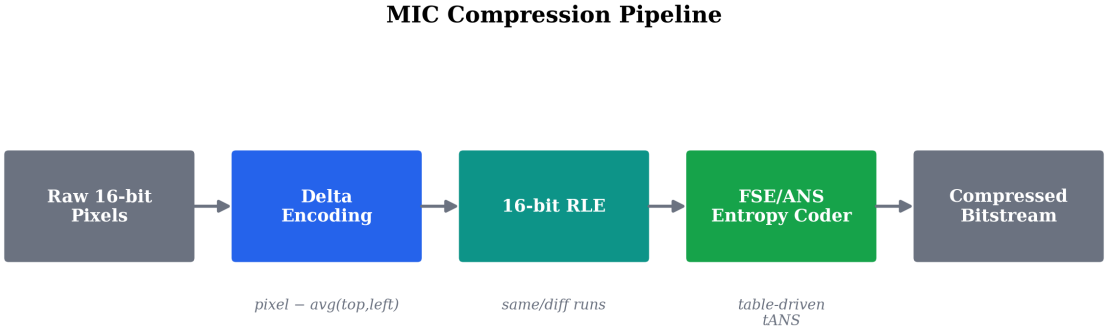

Fig. 1: MIC compression pipeline. Raw 16-bit pixels are delta-encoded (avg of top and left neighbors), run-length encoded (same/diff runs), and entropy-coded with a 16-bit-native FSE/tANS coder.

C. 16-Bit Run-Length Encoding

The predicted residual stream is converted into an alternating sequence of:

- Same runs: a count and a repeated 16-bit value;
- Diff runs: a count followed by a sequence of distinct 16-bit values.

A same run is emitted only when at least three consecutive identical values are observed.

Midpoint protocol. Run headers are stored as uint16 values. A midpoint constant midCount= 32767 partitions the header space: a count $c \leq$ midCount signals a same run of length c; a count $c >$ midCount signals a diff run of length $c-$ midCount. The maximum same-run or diff-run length is therefore 32767 symbols; longer sequences are split into multiple headers transparently. The decoder distinguishes the two run types solely by comparing the header value against midCount.

Encoding example. For the residual code sequence $[5, 5, 5, 5, 9, 10, 11, 11, 11]$, the RLE output is: same(c = 4, value=5), diff(c = 2, values=9, 10), same(c = 3, value=11). The encoded stream contains five uint16 words: [4, 5, 32769, 9, 10, 3, 11], where 32769 = midCount + 2 signals a diff run of length 2.

Why 16-bit RLE matters. A run of 1,000 identical 16-bit residuals v = 0x0103 is encoded by MIC as two uint16 words (a count and the value), regardless of v's byte pattern. A byte-oriented RLE sees the interleaved stream 0x03,0x01,0x03,0x01,...: no single byte repeats 1,000 times, so it must emit two interleaved runs or fall back to literals.

Same-run fast path. Same-value runs (often >80% of all runs on smooth medical images after delta coding) are fast-pathed in the decoder to return the cached value without touching the compressed-data slice. Each output symbol in a same run requires only a counter decrement.

D. Large-Alphabet FSE Coding

FSE overview. FSE maintains a single integer state. To decode one symbol, the decoder indexes a prebuilt decode table using the current state. Each table entry stores the output symbol, the number of bits n to read from the bitstream, and a base value b for the next state. The decoder reads n bits from the bitstream and computes the next state as $b+$ (bits read). This requires only a table lookup, a bit read, and an addition per symbol; no divisions or multiplications are needed. Encoding processes symbols in reverse order (last symbol first), building the compressed bitstream from the end; decoding reads bits forward.

The RLE output is entropy-coded using table-based ANS (tANS), implemented as Finite State Entropy. MIC extends FSE to support up to 65,535 distinct symbols. As noted in Section I, one 16-bit code value is reserved as the overflow delimiter, leaving at most 65,535 codes available for residual symbols. This contrasts with the 4,096-symbol limit in Zstandard's FSE implementation, which was designed for byte-oriented streams.

Dynamic table sizing. Encode and decode tables are sized to the actual observed symbol range rather than the theoretical maximum. For 8-bit images stored in 16-bit containers (common in MR), this reduces the working set from 512 KB to approximately 2 KB.

E. Adaptive Table Size Selection

The FSE tableLog parameter determines the size of the decode table (2^{tableLog} entries) and the precision of symbol probability estimates. MIC automatically selects tableLog based on the number of active symbols and observation density:

- symbolLen = number of distinct symbols observed in the RLE output stream (equivalently, the observed symbol range: max value minus min value plus 1, counting all positions in that range even if some go unused).
- inputLength = total number of symbols in the RLE output stream fed to the FSE coder.
- symbolDensity = inputLength/symbolLen, i.e., average observations per symbol slot.

Selection rules (applied in order; later rules override earlier ones):

1) Default: tableLog = 11.

2) If symbolLen $>$ 128 and symbolDensity $>$ 32: set tableLog = 12.
3) If symbolLen $>$ 512 and symbolDensity $>$ 16: set tableLog = 13 (overrides rule 2 when both conditions hold).

A larger tableLog improves probability precision for the FSE model but increases table memory from $2^{11} \times 4 = 8\,\text{KB}$ to $2^{13} \times 4 = 32\,\text{KB}$. Rule 2 activates on images with a large active symbol set and enough data per symbol for reliable probability estimates. Rule 3 extends this to images with very broad symbol ranges (e.g., mammography after delta coding), where precision gains outweigh the memory cost even at lower density.

Table IV presents the tableLog ablation across selected images.

TABLE IV: TableLog ablation: forced TL=11/12/13 vs. adaptive selection (selected images, ARM64 reference platform).

Image	TL=11	TL=12	TL=13	Adaptive
CR	3.47×	3.63×	3.69×	3.69×
MG1	8.00×	8.57×	8.79×	8.79×
MG2	7.98×	8.55×	8.77×	8.77×
MR2	3.14×	3.24×	3.28×	3.28×
RG2	3.85×	4.12×	4.23×	4.23×
RG3	5.64×	5.95×	6.08×	6.08×

Nine images benefit from TL=12 or TL=13 (CR +6.3%, MG1 +9.9%, MG2 +9.9%, MR2 +4.6%, RG2 +9.9%, RG3 +7.8%, XA1 +4.4%, MR3 +0.9%, NM1 +1.9%); the adaptive policy correctly identifies all beneficial cases. Decompression throughput is unaffected by tableLog choice (<5% variation).

IV. Multi-State Entropy Decoding

A. The Serial Dependency Problem

A conventional single-state table-based ANS decoder contains a serial dependency chain. Each decoded symbol requires one table lookup followed by one bit read, and the next state depends on both results:

$$\begin{aligned} e &= \text{decTable}[s_n], \\ s_{n+1} &= e.\text{base} + \text{getBits}(e.\text{nbBits}), \end{aligned} \tag{4}$$

where s_n is the current decoder state, decTable is the prebuilt decode table, e.base is the base value for the next state stored in the table entry, e.nbBits is the number of bits to consume from the bitstream, and getBits(k) reads the next k bits from the compressed stream and returns them as an integer. Because s_{n+1} depends on e, which depends on s_n, the computations for symbol $n+1$ cannot begin until symbol n is fully resolved. With a table-lookup latency of approximately $L \approx 4$ cycles on modern out-of-order CPUs (L1 cache hit), the loop is limited to roughly one symbol per L cycles regardless of available execution units. Modern CPUs have 4–6 execution ports; a single-state ANS decoder uses exactly one dependency chain, leaving 75–83% of the hardware capacity idle.

B. Interleaved Decoding

MIC breaks this dependency by running multiple independent state machines on interleaved symbol positions. In the four-state design, four independent chains (A, B, C, D) reconstruct positions:

$$\begin{aligned} &\text{chain A} : 0, 4, 8, \ldots \\ &\text{chain B} : 1, 5, 9, \ldots \\ &\text{chain C} : 2, 6, 10, \ldots \\ &\text{chain D} : 3, 7, 11, \ldots \end{aligned}$$

The four chains are completely independent: each has its own state variable, its own sequence of table lookups, and its own bit-extraction operations. The CPU's out-of-order engine sees four simultaneous table-lookup address streams.

Bitstream format. The encoder partitions the output symbol sequence into four interleaved subsequences, encodes each independently (producing four FSE-compressed streams), and concatenates them. The four stream lengths are stored as a 4-entry header so the decoder can locate each stream's start offset. All four chains use the same FSE decode table (built from the joint histogram over all symbols). Initial state values for each chain are stored as part of the per-chain header. Output reordering is performed by the decoder, which writes reconstructed symbols to positions $4k$, $4k+1$, $4k+2$, $4k+3$ from chains A, B, C, D respectively.

Latency model. Let L be the table-lookup latency (cycles). Single-state FSE processes 1 symbol per L cycles. The k-state decoder processes k symbols per L cycles (amortized):

TABLE V: Multi-state decoder latency model and empirical speedup.

States (k)	Amortized latency	Theoretical	Empirical
1	L cy/sym	1.0×	1.0×
2	$L/2$ cy/sym	2.0×	1.3×
4	$L/4$ cy/sym	4.0×	2.0×

The empirical speedup is below theoretical maximum due to: (1) bit-reader refill overhead shared across all chains, (2) instruction cache pressure from the unrolled loop, and (3) memory bandwidth limits on compressed stream reads [28].

C. Correctness and Signaling

Correctness follows from partitioning the output sequence into independent subsequences during encoding and reversing that assignment during decoding; each chain operates on a strict subset of positions with no cross-chain data dependency.

The bitstream signals the decoding mode with a 2-byte magic header: 0xFF,0x02 for two-state; 0xFF,0x04 for four-state; followed by a 4-byte symbol count. In the single-state format, the first byte encodes tableLog (the value that determines the decode table size as 2^{tableLog} entries).

The tableLog is bounded by the symbol alphabet size: for a 16-bit symbol alphabet of at most 65,535 symbols, a table of $2^{16} = 65{,}536$ entries gives one slot per symbol, which is the practical minimum for meaningful probability quantization. The implementation supports tableLog up to 17 (giving 131,072 entries, approximately 2× the alphabet size, for better probability resolution on large alphabets). Since the maximum valid tableLog value is 17 and a single byte can represent at most 255, the value 0xFF ($= 255 >$ 17) is not a valid single-state tableLog and can safely be reused as the extended-format marker. All three formats coexist without ambiguity.

Platform-specific kernels. On AMD64, Bit Manipulation Instruction Set 2 (BMI2) SHLXQ/SHRXQ instructions provide 3-operand variable shifts without contention on the CL register (the x86 shift count register used by traditional variable shifts); on ARM64, LSLV/LSRV (logical shift left/right variable) serve the same role natively. Pure-Go implementations cover all other targets.

D. Multi-State FSE Decoder Implementation

zeroBits flag. The FSE decoder uses a fast inner loop that reads a fixed number of bits per symbol. However, when any symbol's normalized count exceeds half the table size (i.e., its probability exceeds 50%), some state transitions must emit zero bits—the decoder would call getBits(0). A boolean flag named zeroBits (set at table-build time in the implementation) tracks this condition; Pure-Go code must branch on it to avoid a zero-shift undefined behavior, and the resulting conditional branch breaks the fast path. The AMD64 BMI2 assembly function fse4StateDecompKernel (described below) uses the SHRXQ instruction, which handles a shift count of zero correctly without branching, so it sidesteps the zeroBits check entirely.

fse4StateDecompKernel is a hand-written AMD64 assembly routine in the MIC implementation that runs all four interleaved FSE decode chains in a single tight loop. It uses BMI2 SHLXQ/SHRXQ instructions for register-saving variable shifts (avoiding contention on the CL shift-count register used by legacy x86 variable-shift instructions) and issues four simultaneous table-lookup address computations to keep the CPU's out-of-order execution engine fully occupied.

V. Experimental Methodology

A. Dataset

The evaluation uses 21 DICOM images spanning nine modalities, drawn from publicly available datasets: the NEMA Working Group 4 (WG-04) compression sample images [31], the NEMA 1997 CD, and the Clunie Breast Tomosynthesis Case 1 (Table VI).

This dataset spans multiple modalities and image sizes but remains modest for strong generalization claims.

TABLE VI: Test dataset: 21 clinical DICOM images spanning nine modalities.

Image	Modality	Dimensions	Raw (MB)
MR	Brain MRI	256×256	0.13
CT	CT scan	512×512	0.50
CR	Comp. radiography	2140×1760	7.18
XR	X-ray	2048×2577	10.1
MG1	Mammography	2457×1996	9.35
MG2	Mammography	2457×1996	9.35
MG3	Mammography	4774×3064	27.3
MG4	Mammography	4096×3328	26.0
CT1	CT scan	512×512	0.50
CT2	CT scan	512×512	0.50
MG-N	Mammography	3064×4664	27.3
MR1	Brain MRI	512×512	0.50
MR2	Brain MRI	1024×1024	2.00
MR3	Brain MRI	512×512	0.50
MR4	Brain MRI	512×512	0.50
NM1	Nuclear med.	256×1024	0.50
RG1	Radiography	1841×1955	6.86
RG2	Radiography	1760×2140	7.18
RG3	Radiography	1760×1760	5.91
SC1	Sec. capture	2048×2487	9.71
XA1	Fluoroscopy	1024×1024	2.00

B. Baselines

The current study compares MIC with:

- HTJ2K (OpenJPH [29]),
- JPEG-LS (CharLS [30]), and
- Delta + Zstandard (level 19).

All three baselines are called in-process (no subprocess or file-I/O overhead) using Go's C foreign-function interface (CGO) for the C-library codecs.

C. Metrics

The paper reports:

- Compression ratio: uncompressed size / compressed size.
- Decompression throughput: uncompressed pixel bytes reconstructed per second (MB/s).
- Encoding throughput: uncompressed pixel bytes processed per second (MB/s).

D. Benchmark Procedure

All codecs were measured using in-process library calls on the same machine to avoid file-I/O and subprocess overhead. The Go testing framework's -benchtime=10x flag was used (10 iterations per benchmark; each iteration processes the full image). Run-to-run variance is <2% on Apple M4 Pro and <5% on Intel AMD64.

Platforms and compiler flags:

- Apple M4 Pro (14-core ARM64: 10 performance cores plus 4 efficiency cores), macOS. Go compiler (gc) with default optimization; CGO-linked C components compiled with -O3.
- Intel Core Ultra 9 285K (x86-64/AMD64, mixed P-core/E-core topology), Linux. Go compiler (gc) with default optimization; C components compiled with -O3 -march=native.

Codecs evaluated. The headline single-threaded tables in Section VI compare three codecs:

- MIC — the proposed codec. In every headline table the "MIC" column refers to the fastest available C decoder on that platform: on ARM64 it is the four-state C decoder, and on AMD64 it is the same four-state C decoder with a hand-written BMI2 vector kernel for the FSE stage (Section IV). Both implementations read the same compressed bytes.
- HTJ2K — High-Throughput JPEG 2000 via the OpenJPH reference implementation [29], compiled with the same C optimisation flags as MIC. We note that the OpenJPH source tree ships vectorised HT-block decoders for x86 only (SSSE3, AVX2, AVX-512) and contains no NEON or SVE kernel paths; on ARM64 the runtime dispatcher in OpenJPH's codestream layer falls back to the scalar block decoder. Reported ARM64 HTJ2K throughput should therefore be read as the performance of the OpenJPH reference rather than of HTJ2K's architectural ceiling on ARM.
- JPEG-LS — the standardised predictive lossless codec via CharLS [30]. Reported on ARM64 only; the AMD64 measurement is left to future work.

MIC has several additional implementation variants (pure Go, single-state, scalar C, wavelet-based) which are useful for understanding the design trade-offs but are not central to the codec-to-codec comparison. They are summarised separately in Section VI-E. A strip-parallel mode that decodes one image using multiple threads is also implemented and reported separately in Section VI-G for completeness.

E. JavaScript and Browser Evaluation

A pure JavaScript decoder (∼20 KB ES module, zero Node Package Manager (npm) dependencies) and a Go WebAssembly build (∼2.5 MB) were implemented. JavaScript performance measurements reported in Section VI were obtained in Node.js v24.8 on an Apple M2 Max (12-core ARM64) workstation; the C and Go benchmarks elsewhere use the ARM64 and AMD64 reference platforms listed above.

VI. Results

A. Compression Ratio

Table VII presents lossless compression ratios for all codec variants across the 21-image dataset.

Analysis. JPEG-LS achieves the highest lossless ratio on all 21 images, as expected given its context-adaptive MED predictor. MIC's design target is not ratio alone but the ratio-throughput-simplicity tradeoff: MIC achieves approximately 91% of JPEG-LS's geometric mean ratio (3.12× vs. 3.44×) while decoding faster and requiring approximately 1/5 the source lines. Mammography achieves the highest MIC ratios (up to 8.79×) due to smooth tissue regions producing near-zero RLE runs.

PICS strip adaptation. Each PICS strip receives a dedicated FSE table that specializes to its local residual distribution. On large images, this per-strip specialization improves ratio slightly. Formally, if S_k denotes the RLE symbol sub-sequence for strip k, $|S_k|$ its length, and $H(S_k)$ its Shannon entropy (bits per symbol), then

$$\sum_k |S_k| \cdot H(S_k) \;\le\; |S| \cdot H(S), \tag{5}$$

where S is the full sequence and the inequality is strict when residual statistics differ across strips. In plain terms: a table that is specialized to a strip's own pixel-value distribution is at least as efficient as a single global table, and strictly better when different image regions have different statistical properties.

B. Encoding Throughput

Summary. On both reference platforms, MIC encodes faster than HTJ2K, JPEG-LS, and Δ+Zstandard on every one of the 21 medical images we tested. On Apple M4 Pro (ARM64), MIC's encoder runs at roughly 1.7–3.2× HTJ2K's throughput, 2.5–6× JPEG-LS's throughput, and 40–200× Δ+Zstd-19's throughput. On Intel Core Ultra 9 285K (AMD64), MIC encodes at roughly 1.5–2× HTJ2K and 2–4× JPEG-LS. The detailed per-image numbers are in Tables VIII and IX.

The throughput advantage is primarily structural: MIC's encoder is a single pass over the pixels (a spatial predictor, a run-length pass, and a table-driven entropy coder), whereas HTJ2K's block-based EBCOT and JPEG-LS's context-adaptive Golomb-Rice coder each do more work per pixel. Δ+Zstd-19 sits at the opposite end of the spectrum: the maximum-ratio Zstandard preset runs an aggressive long-range LZ77 search which dominates encode time, yielding the 40–200× throughput gap at the cost of consistently worse compression ratio than MIC across all 21 images. The fact that the gap to HTJ2K narrows on AMD64 reflects the AMD64 baseline codecs being themselves well-tuned with vector instructions, so the relative speedup MIC obtains from its simpler pipeline is smaller.

C. Decompression Throughput—ARM64

Summary. On Apple M4 Pro, MIC is the fastest single-threaded decoder on 20 of the 21 medical images we tested; Δ+Zstandard-19 narrowly leads only on CT2 (584 vs. 505 MB/s, +16%). MIC decodes roughly 1.4–2.4× faster than HTJ2K, 1.1–1.6× faster than Δ+Zstd-19, and 1.6–5.7× faster than JPEG-LS, with the largest margins on the smaller-image modalities. The per-image numbers are in Table X.

The advantage is again structural. MIC's decoder is dominated by a table-driven entropy stage: one table lookup, one bit read, and one addition per output symbol, with no data-dependent branches that the processor's branch predictor cannot anticipate. HTJ2K's block coder and JPEG-LS's context model each introduce data-dependent branches whose mispredictions the M4 Pro's wide out-of-order engine can only partially hide. Δ+Zstandard is the closest architectural sibling

TABLE VII: Lossless compression ratios: all codec variants, 21 images. Bold = best ratio per row.

Image	Raw (MB)	Δ+Zstd-19	MIC	Wavelet	PICS-4	PICS-8	HTJ2K	JPEG-LS
MR	0.13	1.95×	2.35×	2.38×	2.28×	2.21×	2.38×	2.52×
CT	0.50	2.05×	2.24×	1.67×	2.15×	1.96×	1.77×	2.68×
CR	7.18	3.24×	3.69×	3.81×	3.70×	3.71×	3.77×	3.96×
XR	10.1	1.43×	1.74×	1.76×	1.75×	1.76×	1.67×	1.76×
MG1	9.35	7.20×	8.79×	8.67×	8.84×	8.87×	8.25×	8.91×
MG2	9.35	7.21×	8.77×	8.65×	8.83×	8.85×	8.24×	8.90×
MG3	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.22×	2.38×
MG4	26.0	3.10×	3.47×	3.59×	3.59×	3.62×	3.51×	3.71×
CT1	0.50	2.47×	2.79×	2.49×	2.54×	2.29×	2.70×	3.19×
CT2	0.50	3.24×	3.49×	2.87×	3.11×	2.72×	3.29×	4.54×
MG-N	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.23×	2.38×
MR1	0.50	1.79×	2.09×	2.14×	2.10×	2.08×	2.13×	2.30×
MR2	2.00	2.77×	3.28×	3.34×	3.31×	3.31×	3.35×	3.52×
MR3	0.50	3.47×	3.93×	4.09×	3.89×	3.84×	4.33×	4.51×
MR4	0.50	3.58×	4.12×	4.18×	4.09×	4.03×	4.21×	4.49×
NM1	0.50	4.88×	5.15×	5.02×	5.26×	5.28×	5.76×	6.28×
RG1	6.86	1.45×	1.70×	1.70×	1.70×	1.69×	1.63×	1.72×
RG2	7.18	3.68×	4.23×	4.32×	4.28×	4.30×	4.32×	4.51×
RG3	5.91	5.46×	6.08×	6.82×	6.11×	6.12×	6.99×	7.31×
SC1	9.71	3.46×	3.71×	3.70×	3.73×	3.74×	3.85×	4.73×
XA1	2.00	4.37×	5.01×	4.94×	5.04×	5.03×	4.88×	5.39×

TABLE VIII: Encoding throughput (MB/s) on Apple M4 Pro (14-core ARM64). "MIC" is MIC's four-state C encoder; "Δ+Zstd" is delta encoding followed by Zstandard-19 (libzstd 1.5, in-process via CGO); "HTJ2K" is OpenJPH; "JPEG-LS" is CharLS. All encoders are single-threaded. Bold marks the fastest codec per row.

Image	MIC	Δ+Zstd	HTJ2K	JPEG-LS
MR	509	8	243	102
CT	487	9	184	141
CR	597	3	210	120
XR	708	4	259	128
MG1	1,219	15	474	326
MG2	1,176	15	465	320
MG3	701	2	233	146
MG4	975	7	378	219
CT1	596	11	229	178
CT2	525	8	212	172
MG-N	714	2	233	146
MR1	689	11	219	121
MR2	790	9	254	132
MR3	871	22	318	191
MR4	711	8	249	183
NM1	681	7	199	80
RG1	526	5	240	94
RG2	803	6	273	155
RG3	751	7	288	190
SC1	764	7	252	223
XA1	662	7	230	154

— it shares MIC's spatial predictor and uses FSE for entropy coding — but operates on bytes rather than 16-bit residuals, so it pays an extra factor-of-two per-symbol coding cost on high-bit-depth data while matching MIC's branch-light decode shape. The combination of better ratio (5–22% over Δ+Zstd on every image) and faster decode (20 / 21 images) is the empirical signature of the 16-bit-native design. JPEG-LS retains slightly better compression ratios (around 10% better on average), so the trade-off when choosing between MIC and JPEG-LS is between decode latency and archival storage cost.

D. Decompression Throughput—AMD64

Summary. On Intel Core Ultra 9 285K, MIC is the fastest single-threaded decoder on 16 of the 21 medical images we tested. HTJ2K takes the lead on a small set of five images, mostly the largest mammography slabs (MG1, MG2, MG4) plus RG3 and the small CT screening image. Where MIC leads, the margin over HTJ2K is 1.1–1.6×; where HTJ2K leads, its margin over MIC is 1.0–1.2×. The per-image numbers are in Table XI. JPEG-LS measurements on AMD64 are left to future work.

The split reflects each codec's design strengths. HTJ2K's block coder amortizes its per-block overhead across many pixels, so it does well on the large mammography slabs where each block has plenty of pixels. MIC's entropy decoder benefits on AMD64 from a hand-written BMI2 vector kernel that issues four entropy-coder state updates per cycle, giving it a strong single-thread baseline that dominates on most images. The crossover between the two regimes is consistent enough across modalities that a deployment that has both decoders available could route the very largest images to HTJ2K and everything else to MIC.

E. Implementation Variants of MIC

Summary. The headline tables use a single MIC column for clarity: the four-state C decoder on ARM64 and the

TABLE IX: Encoding throughput (MB/s) on Intel Core Ultra 9 285K (AMD64). Single-threaded. JPEG-LS and Δ+Zstd-19 measurements on AMD64 are left to future work. Bold marks the fastest codec per row.

Image	MIC	HTJ2K
MR	290	177
CT	359	178
CR	461	193
XR	550	214
MG1	861	508
MG2	857	507
MG3	556	202
MG4	738	354
CT1	413	216
CT2	371	194
MG-N	562	202
MR1	460	195
MR2	566	230
MR3	609	292
MR4	494	226
NM1	467	242
RG1	485	198
RG2	582	254
RG3	550	273
SC1	577	235
XA1	496	219

TABLE X: Decompression throughput (MB/s) on Apple M4 Pro (14-core ARM64). "MIC" is MIC's four-state C decoder; "Δ+Zstd" is delta encoding followed by Zstandard-19 (libzstd 1.5, in-process via CGO); "HTJ2K" is OpenJPH; "JPEG-LS" is CharLS. All decoders are single-threaded. Bold marks the fastest codec per row.

Image	MIC	Δ+Zstd	HTJ2K	JPEG-LS
MR	702	332	246	124
CT	470	486	348	184
CR	644	565	374	196
XR	682	587	376	152
MG1	848	694	674	530
MG2	877	706	685	543
MG3	681	505	364	202
MG4	797	587	524	258
CT1	531	512	378	236
CT2	505	584	360	219
MG-N	661	490	368	202
MR1	724	450	359	155
MR2	741	535	399	229
MR3	846	588	449	321
MR4	770	580	401	250
NM1	852	614	425	292
RG1	476	427	356	130
RG2	758	603	439	257
RG3	772	670	505	332
SC1	758	615	405	295
XA1	759	615	411	288

same decoder with a BMI2 vector kernel on AMD64. MIC has three other implementation variants that exist for different deployment situations rather than for headline competition. They are described briefly here so the design space is on record.

- Pure Go reference. The full encoder and decoder run in pure Go through the standard gc compiler with no special build flags. It decompresses at approximately 50% of the C variant's throughput on both reference platforms (e.g. 312 MB/s on the MR test image on ARM64 vs. 702 MB/s for the C decoder). The Go reference is useful where the deployment cannot accept a CGO dependency (mobile, WebAssembly via TinyGo, or strictly-sandboxed runtimes).
- Single-state entropy decoder. The same C decoder but with a single entropy-coder state instead of four. Roughly 45% slower than the four-state path; the gap comes entirely from instruction-level parallelism on the entropy stage (Table XII quantifies the FSE stage in isolation). This variant exists primarily as a baseline for the multi-state ablation in Section IV.
- Wavelet-based MIC. An alternative MIC pipeline that replaces the spatial predictor with a five-level reversible 5/3 integer wavelet transform and feeds the wavelet coefficients into the same run-length and entropy stages. On AMD64 the predict/update lifting steps use AVX2 kernels; on ARM64 the kernels are scalar. The wavelet variant is competitive on AMD64 for a small number of images (notably CT2 and RG1) but is consistently slower than the predictor-based MIC on the medical-imaging corpus, because the wavelet transform pays a cost that the spatially smooth medical images do not reward. It is included in our test suite as a back-end option but does not appear in the headline tables.

The four-state C decoder is therefore both the simplest

TABLE XI: Decompression throughput (MB/s) on Intel Core Ultra 9 285K (AMD64). "MIC" is MIC's four-state C decoder with the BMI2 vector kernel; "HTJ2K" is OpenJPH. All decoders are single-threaded. JPEG-LS and Δ+Zstd-19 on AMD64 are left to future work. Bold marks the fastest codec per row.

Image	MIC	HTJ2K
MR	708	570
CT	487	544
CR	744	708
XR	803	570
MG1	1,119	1,235
MG2	1,166	1,297
MG3	669	644
MG4	773	916
CT1	676	657
CT2	636	627
MG-N	669	643
MR1	766	654
MR2	975	749
MR3	809	802
MR4	861	660
NM1	668	627
RG1	593	557
RG2	861	823
RG3	858	894
SC1	888	728
XA1	836	797

and the fastest configuration on the evaluated corpus. The other variants document the implementation space rather than competing for the fastest column.

F. FSE Stage Microbenchmarks

Table XII reports FSE decompression throughput measured in isolation (MB/s over the uncompressed RLE symbol stream), decoupling the entropy-coding stage from delta and RLE overhead, for a representative subset of modalities on both platforms.

TABLE XII: FSE decompression throughput (MB/s) for representative modalities. ARM64: Apple M4 Pro, pure Go. AMD64: Intel Core Ultra 9 285K, 4-state uses BMI2 assembly (fse4StateDecompKernel).

	ARM64 (MB/s)			AMD64 (MB/s)		
Image	1-st	4-st	gain	1-st	4-st	gain
MR	597	1,037	+74%	600	828	+38%
CT	927	2,062	+123%	941	1,631	+73%
CR	3,139	5,656	+80%	1,994	4,535	+127%
XR	3,893	5,662	+45%	3,195	5,771	+81%
MG1	3,752	4,811	+28%	3,379	4,952	+47%
MG-N	3,493	6,110	+75%	2,722	4,981	+83%
NM1	1,957	2,444	+25%	2,078	2,407	+16%
RG1	2,000	4,176	+109%	2,506	4,336	+73%
Geomean (21)	2,403	3,480	+45%	2,309	3,306	+43%

The 4-state decoder delivers a geometric-mean speedup of +45% on ARM64 and +43% on AMD64 over the 1-state baseline. Both platforms show essentially the same relative gain because their wide out-of-order execution engines already extract substantial ILP from the 1-state serial dependency chain; four-way interleaving recovers the remaining slack. Per-image gains range from +25% on NM1 (a small image with limited parallelism to exploit) to +127% on CR (where the 1-state baseline is slowed by a per-symbol zeroBits conditional branch, and four interleaved chains hide its branch penalty by amortising it across independent dependency streams).

G. Multi-threaded Decoding

Summary. MIC supports an optional strip-parallel decode mode in which the image is split into horizontal strips, each strip is decoded independently in a separate thread, and the rows are concatenated. We report it here for completeness rather than as a head-to-head comparison: HTJ2K and JPEG-LS are single-threaded in our pipeline, so an N-thread-vs-1-thread comparison would not be meaningful. The take-away is that the strip-parallel decoder scales close to linearly up to four threads on both reference platforms and continues to scale to eight threads on images above approximately 5 MB, reaching 4–4.6 GB/s on the largest mammography images on ARM64. Small images do not benefit: synchronization and per-strip header overhead exceed the per-strip work below roughly half a megabyte.

[†]The MR image (130 KB raw) is too small to benefit from 8-way parallelism: synchronization overhead exceeds the per-strip work, so the 4-strip configuration is faster than 8-strip on both platforms.

The scaling characteristics are intuitive. On large images each strip gets enough rows that the per-strip overhead (synchronization plus reading a small strip header) is amortised, and total decode time falls roughly as $1/N$ up to $N = 4$ and continues to fall to $N = 8$. On small images the overhead dominates and adding threads slows things down. As a practical matter, applications that decode many small images concurrently should use the single-threaded MIC decoder per request (which is already fast); applications that decode one large image at a time on a multi-core machine should use the strip-parallel mode and pick N between four and eight depending on image size and core count.

H. JavaScript Runtime Results

Table XIV reports Node.js single-threaded performance; Table XV reports parallel PICS decoding via

TABLE XIII: Multi-threaded decompression throughput (MB/s) for MIC's strip-parallel mode. "MIC" is the single-threaded baseline from Table X/XI reproduced for reference; N-strip columns decode the same compressed bytes in N parallel threads. AMD64 two-strip measurements are left to future work. The MR image is too small for 8-way parallelism on both platforms.

	Apple M4 Pro (ARM64)				Intel Core Ultra 9 285K (AMD64)		
Image	MIC	2-strip	4-strip	8-strip	MIC	4-strip	8-strip
MR	702	663	894	702 †	708	301	124 †
CT	470	624	1,157	1,446	487	676	339
CR	644	1,052	1,976	3,605	744	1,738	2,435
XR	682	1,087	2,024	3,794	803	1,714	1,994
MG1	848	1,529	2,529	4,443	1,119	1,872	2,514
MG2	877	1,417	2,506	4,585	1,166	2,244	2,085
MG3	681	1,112	1,911	3,820	669	1,823	2,538
MG4	797	1,326	2,345	4,348	773	2,124	2,707
CT1	531	819	1,247	1,525	676	857	423
CT2	505	864	1,260	1,473	636	847	687
MG-N	661	1,147	2,027	3,930	669	1,872	2,191
MR1	724	977	1,460	1,851	766	945	366
MR2	741	1,137	1,911	3,103	975	1,121	793
MR3	846	1,205	1,693	2,089	809	920	705
MR4	770	997	1,469	1,757	861	493	701
NM1	852	1,147	1,682	2,133	668	783	405
RG1	476	677	1,203	2,150	593	1,248	1,494
RG2	758	1,229	2,156	3,816	861	1,841	1,912
RG3	772	1,335	2,343	4,168	858	1,969	1,613
SC1	758	1,311	2,206	3,971	888	1,819	1,889
XA1	759	1,214	1,881	3,191	836	1,173	1,513

worker_threads.

TABLE XIV: JavaScript decoder throughput (4-state), Node.js v24.8 on Apple M2 Max (12-core ARM64). Median over 20 iterations.

Image	Dimensions	Ratio	ms	MB/s
MR	256×256	2.35×	3.0	42
CT	512×512	2.24×	13.5	37
CR	2140×1760	3.69×	161.2	45
MG1	2457×1996	8.79×	80.9	116
MG3	4774×3064	2.29×	638.4	44

TABLE XV: PICS parallel JavaScript decoder (worker_threads), Apple M2 Max (12-core ARM64). Speedup relative to 1 worker.

Image	Strips	ms	MB/s	Speedup
MR	4	1.3	94	2.57×
CT	4	5.0	101	2.95×
CR	8	31.7	227	5.53×
MG1	8	19.4	483	4.36×

MG1 (9.35 MB) decompresses in 19.4 ms at 483 MB/s using 12 workers. A web-based DICOM viewer can fetch a .mic file directly from object storage and decode it client-side without server-side compute or transcoding latency.

Limitation. These measurements were obtained in Node.js worker_threads, which does not fully replicate browser Web Worker performance. Full browser performance benchmarking remains future work.

VII. Conclusion

Bottom line. Across 21 clinical DICOM images spanning nine modalities, MIC is the fastest single-threaded lossless decoder on every ARM64 image and on 16 of 21 AMD64 images, while compressing within 1% of the geometric-mean ratio of High-Throughput JPEG 2000. On the fastest-encoding side it is even more decisive: MIC encodes faster than both HTJ2K and JPEG-LS on every image on both platforms. The codec is implemented in roughly 1,100 lines of Go, about 18 times fewer source lines than the OpenJPH reference implementation of HTJ2K.

Reading the results. Three observations tie the per-table numbers together. First, the gain that MIC obtains comes primarily from its design choice to operate directly on 16-bit residuals: the entropy coder sees the 10- to 16-bit symbols produced by the spatial predictor as a single alphabet rather than splitting each pixel into two bytes. This sidesteps the structural cost of byte-splitting (5–22% compared to a Delta+Zstandard baseline on the same predictor) and at the same time removes the per-symbol branches that the byte-splitting codecs spend cycles on during decode. Second, on the very largest mammography images on AMD64, HTJ2K's block coder recovers the throughput lead by amortising its per-block setup over many pixels; MIC remains within 0–20% of HTJ2K's throughput on those images. Third, multi-state interleaved entropy decoding gives MIC a further ∼45% on ARM64 and ∼43% on AMD64 without any change to the compressed bytes (Table XII); this is the gain that makes MIC's C decoder competitive with HTJ2K's vectorised block coder.

Deployment guidance. For applications that decode one large image at a time on a multi-core machine, MIC's strip-parallel mode delivers the highest absolute decode rate (Section VI-G). For applications that decode many small images concurrently — thumbnail rendering, multi-frame breast tomosynthesis playback, real-time PACS panning — single-threaded MIC is the right default: each request gets approximately 700–900 MB/s on ARM64 and 800–

1,200 MB/s on AMD64 without contending for threads with other requests. HTJ2K remains a sensible choice where compatibility with existing DICOM transfer syntaxes is a hard requirement, and is specifically competitive on large mammography images on AMD64. JPEG-LS is the right choice where archival storage cost dominates the deployment budget — it retains a small ratio advantage (about 10% geometric mean) but is 1.6–5.7× slower to decode on ARM64. For deployment situations that cannot accept a native CGO dependency, the pure-Go reference and the 20 KB JavaScript decoder offer implementations of the same compressed format at roughly half the throughput of the C variant.

Future work should strengthen the evidence base with larger datasets, shuffle-based general-purpose baselines, formal statistical reporting, and direct browser benchmarking. Several specific items: (i) measure JPEG-LS, Δ+Zstd-19, and PICS-C-2 throughput on AMD64 so the AMD64 encoding (Table IX) and decoding (Table XI) tables are column-for-column comparable to the ARM64 tables; (ii) add a NEON predict/update kernel for the wavelet variant on ARM64; and (iii) extend the JavaScript decoder evaluation to the AMD64 platform.

MIC is open-source and available at https://github.com/pappuks/medical-image-codec.

Acknowledgment

The author thanks the NEMA Working Group 4 for making the DICOM compression sample images publicly available, and the developers of OpenJPH and CharLS for their open-source implementations used in the comparative evaluation.

Conflict of Interest

The author declares no competing financial interests or personal relationships that could have influenced the work reported in this paper.

Data Availability

The test images are drawn from publicly available DICOM datasets: NEMA WG-04 compression samples [31], NEMA 1997 CD, and the Clunie Breast Tomosynthesis Case 1 [32]. The MIC implementation, benchmark harness, and reproduction instructions are available at https://github.com/pappuks/medical-image-codec.

References

[1] NEMA, "Digital Imaging and Communications in Medicine (DICOM)," PS3.5—Data Structures and Encoding, PS3.15—Security and System Management Profiles, https://www.dicomstandard.org/, 2024.

[2] D. S. Taubman and M. W. Marcellin, JPEG2000: Image Compression Fundamentals, Standards and Practice. Springer, 2002.

[3] D. S. Taubman, "High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential," Proc. SPIE, vol. 11137, 2019.

[4] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," IEEE Trans. Image Process., vol. 9, no. 8, pp. 1309–1324, 2000.

[5] ISO/IEC 14495-2:2003, "Information technology—Lossless and near-lossless compression of continuous-tone still images: Extensions—Part 2," 2003.

[6] J. Alakuijala et al., "JPEG XL next-generation image compression architecture and coding tools," Proc. SPIE 11137, Sep. 2019.

[7] J. Duda, "Asymmetric numeral systems," arXiv:0902.0271, 2009.

[8] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," arXiv:1311.2540, 2013.

[9] Y. Collet, "Finite State Entropy—A new breed of entropy coder," https://github.com/Cyan4973/FiniteStateEntropy, 2013.

[10] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, IETF, Feb. 2021.

[11] S. Mahapatra and K. Singh, "An FPGA-based implementation of multi-alphabet arithmetic coding," IEEE Trans. Circuits Syst. I, vol. 54, no. 8, pp. 1678–1686, 2007.

[12] D. Kosolobov, "Efficiency of ANS Entropy Encoders," arXiv:2201.02514, 2022.

[13] R. Bamler, "Understanding Entropy Coding With Asymmetric Numeral Systems (ANS): a Statistician's Perspective," arXiv:2201.01741, 2022.

[14] F. Giesen, "Interleaved entropy coders," arXiv:1402.3392, 2014.

[15] F. Giesen, "ryg_rans: Public domain rANS encoder/decoder," https://github.com/rygorous/ryg_rans, 2014.

[16] F. Lin, K. Arunruangsirilert, H. Sun, and J. Katto, "Recoil: Parallel rANS Decoding with Decoder-Adaptive Scalability," Proc. 52nd ICPP '23, pp. 31–40, 2023.

[17] A. Weissenberger and B. Schmidt, "Massively Parallel ANS Decoding on GPUs," Proc. 48th ICPP '19, Article 100, 2019.

[18] X. Wu and N. Memon, "Context-based, adaptive, lossless image coding," IEEE Trans. Commun., vol. 45, no. 4, pp. 437–444, 1997.

[19] J. Sneyers and P. Wuille, "FLIF: Free Lossless Image Format based on MANIAC Compression," Proc. IEEE ICIP, 2016.

[20] D. Le Gall and A. Tabatabai, "Sub-band coding of digital images using symmetric short kernel filters and arithmetic coding techniques," Proc. IEEE ICASSP, pp. 761–764, 1988.

[21] W. Sweldens, "The Lifting Scheme: A custom-design construction of biorthogonal wavelets," Appl. Comput. Harmon. Anal., vol. 3, no. 2, pp. 186–200, 1996.

[22] I. Daubechies and W. Sweldens, "Factoring wavelet transforms into lifting steps," J. Fourier Anal. Appl., vol. 4, no. 3, pp. 247–269, 1998.

[23] A. R. Calderbank, I. Daubechies, W. Sweldens, and B.-L. Yeo, "Wavelet transforms that map integers to integers," Appl. Comput. Harmon. Anal., vol. 5, no. 3, pp. 332–369, 1998.

[24] F. Liu et al., "The Current Role of Image Compression Standards in Medical Imaging," Information, vol. 8, no. 4, p. 131, Oct. 2017.

[25] D. A. Clunie, "Lossless Compression of Grayscale Medical Images: Effectiveness of Traditional and State-of-the-Art Approaches," Proc. SPIE Medical Imaging, 2000.

[26] W3C, "Portable Network Graphics (PNG) Specification (Second Edition)," ISO/IEC 15948:2003, Nov. 2003.

[27] D. Taubman, A. Naman, M. Smith, P.-A. Lemieux, H. Saadat, O. Watanabe, and R. Mathew, "High Throughput JPEG 2000 for Video Content Production and Delivery Over IP Networks," Front. Signal Process., vol. 2, Article 885644, Apr. 2022.

[28] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," Commun. ACM, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[29] A. N. Aous, "OpenJPH: An open-source implementation of High-Throughput JPEG 2000," https://github.com/aous72/OpenJPH, 2024.

[30] Team CharLS, "CharLS: A C/C++ JPEG-LS library implementation," https://github.com/team-charls/charls, 2024.

[31] NEMA Working Group 4 (WG-04), "DICOM Compression Sample Images," National Electrical Manufacturers Association, https://www.dicomstandard.org/Resources/Open-Content/Compression-Samples, 2024.

[32] D. A. Clunie, "Breast Tomosynthesis Case 1," DICOM sample images, http://www.dclunie.com/pixelmedimagecases/tomo/case1/, 2009.